mdflux Testing Strategy

Overview

This document defines a comprehensive, use case driven testing strategy for mdflux - a CLI utility and Go library for converting CommonMark Markdown into HTML5/XHTML or PDF documents.

Testing Philosophy

- **Use Case Driven**: Tests are organized around real-world usage scenarios
- **Behavior Verification**: Focus on validating expected outcomes rather than implementation details
- **Regression Prevention**: Ensure changes don't break existing functionality
- **Coverage Breadth**: Test all critical paths while avoiding redundant tests

---

1. Unit Testing

1.1 Configuration Package (`internal/pkg/mdflux/config`)

Use Case	Test Scenario	Expected Outcome
UC-CFG-001	Load default configuration with no files present	All defaults applied correctly
UC-CFG-002	Load configuration from `./mdflux.cfg.toml`	Local config values override defaults
UC-CFG-003	Load configuration from user config directory	User config loaded when local not present
UC-CFG-004	Load configuration from custom path via `-c` flag	Custom config path takes precedence
UC-CFG-005	CLI flags override config file values	Flag values supersede file values
UC-CFG-006	Environment variables override config file	Env vars respected in precedence chain
UC-CFG-007	Invalid config file path provided	Graceful error with clear message

Use Case	Test Scenario	Expected Outcome
UC-CFG-008	Malformed TOML syntax in config file	Parse error reported with location
UC-CFG-009	Unknown configuration keys in file	Ignored without error (forward compatibility)
UC-CFG-010	All extension toggles function independently	Each extension can be enabled/disabled

Test Data Requirements:

- Valid TOML config files with various option combinations
- Invalid/malformed TOML files for error handling tests
- Environment variable fixtures

1.2 Converter Package (`internal/pkg/mdflux/converter`)

Core Conversion

Use Case	Test Scenario	Expected Outcome
UC-CNV-001	Convert empty markdown input	Valid HTML with empty body
UC-CNV-002	Convert plain text paragraph	Single `<p>` element with text
UC-CNV-003	Convert multiple paragraphs	Correct paragraph separation
UC-CNV-004	Convert headings (h1-h6)	Proper heading hierarchy
UC-CNV-005	Convert inline formatting (bold, italic, code)	Correct inline elements
UC-CNV-006	Convert blockquotes	Nested `<blockquote>` elements
UC-CNV-007	Convert ordered lists	`<ol>` with `<li>` items
UC-CNV-008	Convert unordered lists	`<ul>` with `<li>` items

Use Case	Test Scenario	Expected Outcome
UC-CNV-009	Convert nested lists	Proper nesting structure
UC-CNV-010	Convert code blocks with language	`<pre><code class="language-X">`
UC-CNV-011	Convert horizontal rules	`<hr>` element
UC-CNV-012	Convert images	`<img>` with src and alt
UC-CNV-013	Convert links	`<a>` with href
UC-CNV-014	Convert links with titles	`<a>` with title attribute
UC-CNV-015	Handle unsafe HTML when disabled	HTML escaped in output
UC-CNV-016	Allow unsafe HTML when enabled	Raw HTML passed through
UC-CNV-017	Hard wraps option converts newlines	` ` tags inserted
UC-CNV-018	XHTML output mode	Self-closing tags, XML declaration

Extension Testing

Tables Extension:

Use Case	Test Scenario	Expected Outcome
UC-EXT-TBL-001	Simple table with header	`<table>` with `<thead>` and `<tbody>`
UC-EXT-TBL-002	Table with alignment	`style="text-align:X"` on cells
UC-EXT-TBL-003	Table without header row	Proper structure maintained
UC-EXT-TBL-004	Table with varying column counts	Graceful handling

Use Case	Test Scenario	Expected Outcome
UC-EXT-TBL-005	Table disabled via config	Raw markdown in output

Strikethrough Extension:

Use Case	Test Scenario	Expected Outcome
UC-EXT-STR-001	`~~text~~` converted	`<del>` or `<s>` element
UC-EXT-STR-002	Nested strikethrough with other formatting	Correct nesting order
UC-EXT-STR-003	Extension disabled	Raw `~~` preserved

Linkify Extension:

Use Case	Test Scenario	Expected Outcome
UC-EXT-LNK-001	Plain URL auto-linked	`<a href="...">` created
UC-EXT-LNK-002	Email address auto-linked	`mailto:` link created
UC-EXT-LNK-003	URL within code block	Not linked (code preserved)
UC-EXT-LNK-004	Extension disabled	URLs remain plain text

Task List Extension:

Use Case	Test Scenario	Expected Outcome
UC-EXT-TSK-001	`- [ ] item` converted	Unchecked checkbox
UC-EXT-TSK-002	`- [x] item` converted	Checked checkbox
UC-EXT-TSK-003	Mixed regular and task items	Correct differentiation
UC-EXT-TSK-004	Nested task lists	Proper nesting with checkboxes

Definition List Extension:

Use Case	Test Scenario	Expected Outcome
UC-EXT-DEF-001	Term and definition	`<dl>`, `<dt>`, `<dd>` structure
UC-EXT-DEF-002	Multiple definitions per term	Multiple `<dd>` elements

Use Case	Test Scenario	Expected Outcome
UC-EXT-DEF-003	Consecutive definition lists	Properly separated

Footnote Extension:

Use Case	Test Scenario	Expected Outcome
UC-EXT-FN-001	Inline footnote reference	Superscript link created
UC-EXT-FN-002	Footnote definition	Footnote section generated
UC-EXT-FN-003	Multiple footnotes	Correct numbering and linking
UC-EXT-FN-004	Footnote without definition	Graceful handling
UC-EXT-FN-005	Definition without reference	Not rendered

Typographer Extension:

Use Case	Test Scenario	Expected Outcome
UC-EXT-TYP-001	`"text"` converted	Smart quotes applied
UC-EXT-TYP-002	`--` converted	En-dash
UC-EXT-TYP-003	`---` converted	Em-dash
UC-EXT-TYP-004	`...` converted	Ellipsis character
UC-EXT-TYP-005	Inside code blocks	Not converted

CJK Extension:

Use Case	Test Scenario	Expected Outcome
UC-EXT-CJK-001	Chinese text line breaks	Appropriate break handling
UC-EXT-CJK-002	Japanese text line breaks	Appropriate break handling
UC-EXT-CJK-003	Korean text line breaks	Appropriate break handling
UC-EXT-CJK-004	Mixed CJK and Latin text	Correct spacing behavior

KaTeX Extension:

Use Case	Test Scenario	Expected Outcome
UC-EXT-KTX-001	Inline math `$E=mc^2$`	KaTeX rendered inline
UC-EXT-KTX-002	Display math `$$\int_0^1 x^2 dx$$`	Block-level KaTeX
UC-EXT-KTX-003	Invalid LaTeX syntax	Error handling/fallback
UC-EXT-KTX-004	Math in code block	Not processed
UC-EXT-KTX-005	Complex nested expressions	Correct rendering

Mermaid Extension:

Use Case	Test Scenario	Expected Outcome
UC-EXT-MRM-001	Flowchart diagram	SVG embedded in output
UC-EXT-MRM-002	Sequence diagram	SVG embedded in output
UC-EXT-MRM-003	Class diagram	SVG embedded in output
UC-EXT-MRM-004	Gantt chart	SVG embedded in output
UC-EXT-MRM-005	Invalid mermaid syntax	Graceful error handling
UC-EXT-MRM-006	Multiple diagrams in document	Each rendered independently
UC-EXT-MRM-007	Server-side rendering disabled	`<div class="mermaid">` fallback

D2 Extension:

Use Case	Test Scenario	Expected Outcome
UC-EXT-D2-001	Simple D2 diagram	SVG rendered
UC-EXT-D2-002	D2 with Dagre layout	Correct layout applied
UC-EXT-D2-003	D2 with ELK layout	Correct layout applied
UC-EXT-D2-004	D2 with styling	Styles applied to SVG
UC-EXT-D2-005	Invalid D2 syntax	Error message in output

1.3 Template Package (`internal/pkg/mdflux/converter`)

Use Case	Test Scenario	Expected Outcome
UC-TPL-001	Load templates from embedded FS	All templates accessible
UC-TPL-002	HTML5 header template execution	Valid HTML5 DOCTYPE
UC-TPL-003	XHTML header template execution	Valid XHTML 1.0 Strict
UC-TPL-004	Light theme class applied	`class="light"` on body
UC-TPL-005	Dark theme class applied	`class="dark"` on body
UC-TPL-006	Auto theme (no class)	System preference respected
UC-TPL-007	KaTeX CSS included	Stylesheet embedded
UC-TPL-008	Custom title in template	Title rendered in `<head>`

1.4 Mermaid Package (`internal/pkg/mdflux/mermaid`)

Use Case	Test Scenario	Expected Outcome
UC-MRM-001	Renderer initialization	Chrome browser launched
UC-MRM-002	Single diagram rendering	SVG returned
UC-MRM-003	Multiple sequential renders	Browser reused
UC-MRM-004	Concurrent render requests	Mutex prevents race conditions
UC-MRM-005	Chrome not available	Clear error message
UC-MRM-006	Temporary file cleanup	No orphaned temp files
UC-MRM-007	Renderer close	Browser and temp files cleaned

1.5 PDF Package (`internal/pkg/mdflux/pdf`)

Use Case	Test Scenario	Expected Outcome
UC-PDF-001	Generate PDF from HTML file	Valid PDF binary
UC-PDF-002	A4 paper size	Correct dimensions
UC-PDF-003	Letter paper size	Correct dimensions
UC-PDF-004	Legal paper size	Correct dimensions

Use Case	Test Scenario	Expected Outcome
UC-PDF-005	Portrait orientation	Correct orientation
UC-PDF-006	Landscape orientation	Correct orientation
UC-PDF-007	Custom margins	Margins applied
UC-PDF-008	Scale factor applied	Content scaled
UC-PDF-009	Background colors printed	`printBackground: true`
UC-PDF-010	Chrome auto-detection	Browser found and used
UC-PDF-011	Custom Chrome path	Specified path used
UC-PDF-012	Invalid HTML file	Graceful error
UC-PDF-013	Chrome unavailable	Clear error message

2. Integration Testing

2.1 Configuration to Converter Integration

Use Case	Test Scenario	Expected Outcome
UC-INT-001	Config extensions affect converter	Disabled extensions not processed
UC-INT-002	Theme setting applied to output	Correct theme class in HTML
UC-INT-003	XHTML setting produces valid XHTML	Valid XHTML 1.0 Strict output
UC-INT-004	Unsafe HTML setting respected	HTML handling matches config

2.2 Converter to Mermaid Integration

Use Case	Test Scenario	Expected Outcome
UC-INT-010	Document with mermaid code block	Mermaid renderer invoked
UC-INT-011	Document without mermaid	Renderer not initialized
UC-INT-012	Multiple mermaid blocks	All rendered, renderer reused

2.3 HTML to PDF Pipeline

Use Case	Test Scenario	Expected Outcome
UC-INT-020	Convert MD with images to PDF	Images rendered in PDF
UC-INT-021	Convert MD with mermaid to PDF	SVG diagrams in PDF
UC-INT-022	Convert MD with KaTeX to PDF	Math expressions in PDF
UC-INT-023	Temporary HTML file management	Created and cleaned up
UC-INT-024	Large document conversion	Completes without timeout

3. End-to-End (E2E) Testing

3.1 CLI Interface Testing

Use Case	Test Scenario	Expected Outcome
UC-E2E-001	`mdflux -i input.md -o output.html`	HTML file created
UC-E2E-002	`mdflux -i input.md -o output.pdf -f pdf`	PDF file created
UC-E2E-003	`cat input.md | mdflux`	HTML to stdout
UC-E2E-004	`mdflux -i input.md`	HTML to stdout
UC-E2E-005	`mdflux < input.md > output.html`	Stream I/O works
UC-E2E-006	`mdflux -t dark -i input.md`	Dark theme applied
UC-E2E-007	`mdflux -c custom.toml -i input.md`	Custom config loaded
UC-E2E-008	`mdflux -l debug -i input.md`	Debug logging enabled
UC-E2E-009	`mdflux --log_file app.log -i input.md`	Logs written to file
UC-E2E-010	`mdflux -?` or `mdflux --help`	Help text displayed
UC-E2E-011	`mdflux -i nonexistent.md`	Error: file not found
UC-E2E-012	`mdflux -o /readonly/path`	Error: permission denied
UC-E2E-013	`mdflux -f invalid`	Error: unknown format

3.2 Real Document Conversion

Use Case	Test Scenario	Expected Outcome
UC-E2E-020	Convert `docs/demo.md` to HTML	Matches reference output
UC-E2E-021	Convert `test/diagrams-test.md`	All diagrams rendered
UC-E2E-022	Convert `test/katex-examples.md`	All math rendered
UC-E2E-023	Convert `test/d2-diagrams-test.md`	D2 diagrams rendered
UC-E2E-024	Convert document with all extensions	All features work together

3.3 Output Validation

Use Case	Test Scenario	Expected Outcome
UC-E2E-030	HTML output validates	W3C HTML5 compliant
UC-E2E-031	XHTML output validates	XHTML 1.0 Strict compliant
UC-E2E-032	PDF is valid	PDF/A compliant structure
UC-E2E-033	SVG diagrams are valid	SVG 1.1 compliant

---

4. Edge Case Testing

4.1 Input Edge Cases

Use Case	Test Scenario	Expected Outcome
UC-EDGE-001	Empty input file	Valid HTML with empty body
UC-EDGE-002	Very large file (>10MB)	Completes without OOM
UC-EDGE-003	Binary file as input	Graceful handling
UC-EDGE-004	File with only whitespace	Valid HTML
UC-EDGE-005	File with BOM marker	BOM stripped, content processed
UC-EDGE-006	Mixed line endings (CRLF/LF)	Normalized handling
UC-EDGE-007	UTF-8 with various scripts	All characters preserved
UC-EDGE-008	Invalid UTF-8 sequences	Graceful error or replacement

4.2 Markdown Edge Cases

Use Case	Test Scenario	Expected Outcome
UC-EDGE-010	Deeply nested lists (10+ levels)	All levels rendered
UC-EDGE-011	Very long single line	Line handled without truncation
UC-EDGE-012	Code block with triple backticks inside	Proper escaping
UC-EDGE-013	Markdown special chars escaped	Rendered literally
UC-EDGE-014	Reference links at document end	Links resolved
UC-EDGE-015	Circular footnote references	No infinite loop
UC-EDGE-016	Conflicting formatting markers	Predictable resolution

4.3 Diagram Edge Cases

Use Case	Test Scenario	Expected Outcome
UC-EDGE-020	Very large mermaid diagram	Completes or times out gracefully
UC-EDGE-021	Mermaid with special characters	Characters escaped in SVG
UC-EDGE-022	D2 with unicode labels	Unicode preserved
UC-EDGE-023	Empty diagram code block	Empty SVG or placeholder
UC-EDGE-024	Diagram with XSS attempt	Sanitized output

4.4 PDF Edge Cases

Use Case	Test Scenario	Expected Outcome
UC-EDGE-030	Very long document to PDF	Proper pagination
UC-EDGE-031	Document with page breaks	Breaks respected
UC-EDGE-032	Wide content (tables/code)	Overflow handling
UC-EDGE-033	High-resolution images	Scaled appropriately

---

5. Performance Testing

5.1 Benchmarks

Use Case	Metric	Target
UC-PERF-001	Simple 1KB markdown to HTML	< 10ms
UC-PERF-002	Complex 100KB markdown to HTML	< 500ms
UC-PERF-003	Document with 10 mermaid diagrams	< 10s
UC-PERF-004	1MB markdown to HTML	< 2s
UC-PERF-005	HTML to PDF conversion	< 5s
UC-PERF-006	Memory usage for 10MB document	< 500MB

5.2 Concurrency Testing

Use Case	Test Scenario	Expected Outcome
UC-PERF-010	Parallel conversion requests	No race conditions
UC-PERF-011	Mermaid renderer under load	Mutex handles contention
UC-PERF-012	PDF generation parallel requests	Chrome instances managed

5.3 Resource Management

Use Case	Test Scenario	Expected Outcome
UC-PERF-020	Repeated conversions (1000x)	No memory leak
UC-PERF-021	Chrome process lifecycle	Processes cleaned up
UC-PERF-022	Temporary file accumulation	Files cleaned on close

6. Security Testing

6.1 Input Sanitization

Use Case	Test Scenario	Expected Outcome
UC-SEC-001	XSS in markdown content	Sanitized when unsafe disabled

Use Case	Test Scenario	Expected Outcome
UC-SEC-002	Script injection via diagram	Scripts not executed
UC-SEC-003	Path traversal in file input	Rejected
UC-SEC-004	Command injection via config	Escaped/rejected

6.2 Resource Limits

Use Case	Test Scenario	Expected Outcome
UC-SEC-010	Zip bomb markdown (decompression)	N/A - no compression support
UC-SEC-011	Billion laughs (entity expansion)	N/A - no XML entity support
UC-SEC-012	ReDoS in regex patterns	Patterns bounded

---

7. Regression Testing

7.1 Golden File Tests

Maintain a suite of markdown inputs with expected HTML/PDF outputs:

```
test/
├── golden/
│   ├── basic.md          → basic.html
│   ├── tables.md         → tables.html
│   ├── code-blocks.md    → code-blocks.html
│   ├── footnotes.md      → footnotes.html
│   ├── mermaid.md        → mermaid.html
│   ├── katex.md          → katex.html
│   ├── d2.md             → d2.html
│   ├── all-features.md   → all-features.html
│   └── complex-doc.md    → complex-doc.html
```

7.2 Backward Compatibility

Use Case	Test Scenario	Expected Outcome
UC-REG-001	Old config file format	Still parsed correctly
UC-REG-002	Previous CLI flag syntax	Flags still work

Use Case	Test Scenario	Expected Outcome
UC-REG-003	Deprecated options	Warning + fallback

8. Technology Stack by Test Type

8.1 Unit Testing Stack

Component	Technology	Purpose	Go Module
Test Framework	Go `testing`	Core test execution	stdlib
Assertions	`testify/assert`	Readable assertions, comparisons	`github.com/stretchr/testify`
Requirements	`testify/require`	Fatal assertions (stop on failure)	`github.com/stretchr/testify`
Mocking	`testify/mock`	Interface mocking for dependencies	`github.com/stretchr/testify`
Mock Generation	`mockery`	Auto-generate mock implementations	`github.com/vektra/mockery/v2`
Table Tests	`t.Run()` subtests	Parameterized test cases	stdlib
Test Fixtures	`go:embed`	Embed test data files	stdlib
Temp Files	`t.TempDir()`	Auto-cleanup temp directories	stdlib
Parallel Tests	`t.Parallel()`	Concurrent test execution	stdlib

Example Usage:

```
import (
    "testing"
    "github.com/stretchr/testify/assert"
```

```
    "github.com/stretchr/testify/require"
)

func TestConverterBasic(t *testing.T) {
    t.Parallel()

    tests := []struct {
        name     string
        input    string
        expected string
    }{
        {"empty input", "", "<body></body>"},
        {"paragraph", "hello", "<p>hello</p>"},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            result, err := Convert(tt.input)
            require.NoError(t, err)
            assert.Contains(t, result, tt.expected)
        })
    }
}
```

Mocking Chrome Dependencies:

```
//go:generate mockery --name=BrowserRenderer --output=mocks
type BrowserRenderer interface {
    RenderDiagram(code string) (string, error)
    Close() error
}
```

8.2 Integration Testing Stack

Component	Technology	Purpose	Go Module
Test Framework	Go `testing`	Core test execution	stdlib
Build Tags	`//go:build integration`	Separate integration tests	stdlib

Component	Technology	Purpose	Go Module
Test Containers	`testcontainers-go`	Spin up Chrome in Docker	`github.com/testcontainers/testcontainers-go`
Chrome Control	`chromedp`	Direct Chrome interaction	`github.com/chromedp/chromedp`
HTTP Testing	`httptest`	Mock HTTP servers if needed	stdlib `net/http/httptest`
File Comparison	`go-cmp`	Deep equality with diffs	`github.com/google/go-cmp`
Test Setup	`TestMain`	Global setup/teardown	stdlib

Example Usage:

```
//go:build integration

package integration

import (
    "context"
    "testing"
    "github.com/testcontainers/testcontainers-go"
    "github.com/testcontainers/testcontainers-go/modules/chromium"
)

func TestMain(m *testing.M) {
    // Setup: Start Chrome container
    ctx := context.Background()
    container, _ := chromium.Run(ctx, "chromedp/headless-shell:latest")
    defer container.Terminate(ctx)

    os.Exit(m.Run())
}

func TestMermaidRendering(t *testing.T) {
    // Test with real Chrome instance
}
```

8.3 End-to-End (E2E) Testing Stack

Component	Technology	Purpose	Go Module
Test Framework	Go `testing`	Core test execution	stdlib
CLI Execution	`os/exec`	Run mdflux binary	stdlib
Process Control	`exec.CommandContext`	Timeout handling	stdlib
File Operations	`os`, `io`	Read/write test files	stdlib
Assertions	`testify/assert`	Output validation	`github.com/stretchr/testify`
Golden Files	`goldie`	Snapshot testing	`github.com/sebdah/goldie/v2`
HTML Validation	`golang.org/x/net/html`	Parse/validate HTML	`golang.org/x/net/html`
PDF Validation	`pdfcpu`	Validate PDF structure	`github.com/pdfcpu/pdfcpu`

Example Usage:

```go
//go:build e2e

package e2e

import (
    "os/exec"
    "testing"
    "github.com/sebdah/goldie/v2"
    "github.com/stretchr/testify/require"
)

func TestCLIHTMLOutput(t *testing.T) {
    g := goldie.New(t, goldie.WithFixtureDir("testdata/golden"))

    cmd := exec.Command("./mdflux", "-i", "testdata/input.md")
    output, err := cmd.Output()
    require.NoError(t, err)
```

```
    g.Assert(t, "basic-html", output)
}

func TestCLIPDFOutput(t *testing.T) {
    tmpDir := t.TempDir()
    outFile := filepath.Join(tmpDir, "output.pdf")

    cmd := exec.Command("./mdflux", "-i", "testdata/input.md", "-o", outFile, "-
    err := cmd.Run()
    require.NoError(t, err)

    // Validate PDF
    _, err = pdfcpu.ReadFile(outFile, nil)
    require.NoError(t, err)
}
```

8.4 Performance Testing Stack

Component	Technology	Purpose	Go Module
Benchmarks	`testing.B`	Standard Go benchmarks	stdlib
Memory Profiling	`b.ReportAllocs()`	Track allocations	stdlib
CPU Profiling	`runtime/pprof`	CPU profiling	stdlib
Memory Profiling	`runtime/pprof`	Heap analysis	stdlib
Benchmark Stats	`benchstat`	Compare benchmark runs	`golang.org/x/perf/cmd/benchstat`
Load Testing	Custom goroutines	Concurrent load simulation	stdlib
Metrics	`expvar` or custom	Runtime metrics collection	stdlib

Example Usage:

```go
func BenchmarkConvertSimple(b *testing.B) {
    input := []byte("# Hello\n\nWorld")
    b.ResetTimer()
    b.ReportAllocs()

    for i := 0; i < b.N; i++ {
        _, _ = Convert(input)
    }
}

func BenchmarkConvertWithMermaid(b *testing.B) {
    input := loadFixture("mermaid-doc.md")

    b.Run("single", func(b *testing.B) {
        for i := 0; i < b.N; i++ {
            _, _ = ConvertWithMermaid(input)
        }
    })

    b.Run("parallel", func(b *testing.B) {
        b.RunParallel(func(pb *testing.PB) {
            for pb.Next() {
                _, _ = ConvertWithMermaid(input)
            }
        })
    })
}
```

Execution:

```bash
# Run benchmarks
go test -bench=. -benchmem ./...

# Compare before/after
go test -bench=. -count=10 ./... > old.txt
# Make changes
go test -bench=. -count=10 ./... > new.txt
benchstat old.txt new.txt

# CPU profile
go test -bench=BenchmarkConvert -cpuprofile=cpu.prof
go tool pprof cpu.prof

# Memory profile
```

```
go test -bench=BenchmarkConvert -memprofile=mem.prof
go tool pprof mem.prof
```

8.5 Security Testing Stack

Component	Technology	Purpose	Go Module / Tool
Static Analysis	`gosec`	Security vulnerability scanning	`github.com/securego/gosec/v2`
Dependency Audit	`govulncheck`	Known vulnerability detection	`golang.org/x/vuln/cmd/govulncheck`
Fuzzing	Go native fuzzing	Input mutation testing	stdlib `testing.F`
SAST	`semgrep`	Pattern-based security rules	External tool
Input Validation	Custom tests	XSS, injection testing	stdlib
HTML Sanitization	`bluemonday`	Verify HTML sanitization	`github.com/microcosm-cc/bluemonday`

Example Fuzz Testing:

```
func FuzzMarkdownConvert(f *testing.F) {
    // Seed corpus
    f.Add([]byte("# Hello"))
    f.Add([]byte("```mermaid\ngraph TD\n```"))
    f.Add([]byte("<script>alert('xss')</script>"))

    f.Fuzz(func(t *testing.T, input []byte) {
        result, err := Convert(input)
        if err != nil {
            return // Valid to fail on bad input
        }

        // Security assertions
        if strings.Contains(result, "<script>") {
            t.Error("XSS: script tag in output")
        }
```

```
    })
}
```

Execution:

```
# Run gosec
gosec ./...

# Check dependencies
govulncheck ./...

# Run fuzz tests
go test -fuzz=FuzzMarkdownConvert -fuzztime=60s ./...
```

8.6 Regression Testing Stack (Golden Files)

Component	Technology	Purpose	Go Module
Golden File Framework	`goldie`	Snapshot comparison	`github.com/sebdah/goldie/v2`
Diff Generation	`go-cmp`	Detailed diff output	`github.com/google/go-cmp`
Update Mode	`-update` flag	Regenerate golden files	`goldie` flag
File Embedding	`go:embed`	Embed expected outputs	stdlib
Normalization	Custom functions	Normalize timestamps, IDs	Custom

Example Usage:

```
import "github.com/sebdah/goldie/v2"

func TestGoldenHTML(t *testing.T) {
    g := goldie.New(
        t,
        goldie.WithFixtureDir("testdata/golden"),
        goldie.WithNameSuffix(".html"),
        goldie.WithDiffEngine(goldie.ColoredDiff),
    )
```

```
    tests := []string{"basic", "tables", "code-blocks", "mermaid", "katex"}

    for _, name := range tests {
        t.Run(name, func(t *testing.T) {
            input := loadFixture(name + ".md")
            result, _ := Convert(input)

            // Normalize dynamic content (timestamps, random IDs)
            normalized := normalizeOutput(result)

            g.Assert(t, name, []byte(normalized))
        })
    }
}
```

Update Golden Files:

```
go test -v ./... -update
```

8.7 Code Coverage Stack

Component	Technology	Purpose	Go Module / Tool
Coverage Collection	`go test -cover`	Line coverage	stdlib
Coverage Report	`go tool cover`	HTML report generation	stdlib
Coverage Analysis	`gocov`	JSON coverage data	`github.com/axw/gocov`
Coverage Visualization	`gocov-html`	Enhanced HTML reports	`github.com/matm/gocov-html`
CI Integration	`codecov` / `coveralls`	Coverage tracking service	External service
Branch Coverage	`gobco`	Branch coverage analysis	`github.com/rillig/gobco`

Execution:

```
# Generate coverage
go test -coverprofile=coverage.out -covermode=atomic ./...
```

```
# View in terminal
go tool cover -func=coverage.out

# Generate HTML report
go tool cover -html=coverage.out -o coverage.html

# With gocov for detailed analysis
gocov convert coverage.out | gocov-html > coverage-detailed.html
```

8.8 CI/CD Testing Stack

Component	Technology	Purpose
CI Platform	GitHub Actions	Automated test execution
Matrix Testing	Actions matrix	Test on multiple Go versions
Caching	Actions cache	Cache Go modules
Chrome for E2E	`setup-chrome` action	Install headless Chrome
Coverage Upload	`codecov-action`	Upload coverage reports
Linting	`golangci-lint`	Code quality checks
Release Testing	`goreleaser`	Cross-platform build testing

Example GitHub Actions Workflow:

```
name: Tests
on: [push, pull_request]

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        go-version: ['1.22', '1.23']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with:
          go-version: ${{ matrix.go-version }}
      - name: Run unit tests
```

```
      run: go test -race -coverprofile=coverage.out ./...
    - uses: codecov/codecov-action@v4
      with:
        files: coverage.out

  integration-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
      - uses: browser-actions/setup-chrome@v1
      - name: Run integration tests
        run: go test -tags=integration ./test/integration/...

  e2e-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
      - uses: browser-actions/setup-chrome@v1
      - name: Build binary
        run: go build -o mdflux ./cmd/mdflux
      - name: Run E2E tests
        run: go test -tags=e2e ./test/e2e/...

  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: securego/gosec@master
      - name: Run govulncheck
        run: |
          go install golang.org/x/vuln/cmd/govulncheck@latest
          govulncheck ./...

  benchmarks:
    runs-on: ubuntu-latest
    if: github.event_name == 'pull_request'
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
      - name: Run benchmarks
        run: go test -bench=. -benchmem ./... | tee benchmark.txt
      - name: Upload benchmark results
        uses: actions/upload-artifact@v4
        with:
```

```
          name: benchmarks
          path: benchmark.txt
```

---

9. Technology Stack Summary

Test Type	Primary Stack	Key Dependencies
Unit	`testing` + `testify` + `mockery`	`github.com/stretchr/testify`, `github.com/vektra/mockery/v2`
Integration	`testing` + `testcontainers-go` + `go-cmp`	`github.com/testcontainers/testcontainers-go`, `github.com/google/go-cmp`
E2E	`testing` + `os/exec` + `goldie` + `pdfcpu`	`github.com/sebdah/goldie/v2`, `github.com/pdfcpu/pdfcpu`
Performance	`testing.B` + `benchstat` + `pprof`	`golang.org/x/perf/cmd/benchstat`
Security	`testing.F` + `gosec` + `govulncheck`	`github.com/securego/gosec/v2`, `golang.org/x/vuln/cmd/govulncheck`
Regression	`goldie` + `go-cmp`	`github.com/sebdah/goldie/v2`, `github.com/google/go-cmp`
Coverage	`go tool cover` + `gocov`	`github.com/axw/gocov`
CI/CD	GitHub Actions + `golangci-lint`	External services

---

10. Recommended `go.mod` Test Dependencies

```
require (
    // Assertions and mocking
    github.com/stretchr/testify v1.9.0

    // Golden file testing
    github.com/sebdah/goldie/v2 v2.5.5
```

```
    // Deep comparison with diffs
    github.com/google/go-cmp v0.6.0

    // PDF validation
    github.com/pdfcpu/pdfcpu v0.8.0

    // Container-based testing
    github.com/testcontainers/testcontainers-go v0.31.0

    // HTML sanitization verification
    github.com/microcosm-cc/bluemonday v1.0.26
)
```

Development Tools (go install):

```
# Mock generation
go install github.com/vektra/mockery/v2@latest

# Security scanning
go install github.com/securego/gosec/v2/cmd/gosec@latest
go install golang.org/x/vuln/cmd/govulncheck@latest

# Benchmark analysis
go install golang.org/x/perf/cmd/benchstat@latest

# Coverage analysis
go install github.com/axw/gocov/gocov@latest
go install github.com/matm/gocov-html/cmd/gocov-html@latest

# Linting
go install github.com/golangci/golangci-lint/cmd/golangci-lint@latest
```

---

11. Test Implementation Guidelines

11.1 Directory Structure

```
mdflux/
├── internal/pkg/mdflux/
│   ├── config/
│   │   └── config_test.go
│   ├── converter/
│   │   ├── converter_test.go
```

```
│   │   └── templates_test.go
│   ├── mermaid/
│   │   ├── extension_test.go
│   │   └── renderer_test.go
│   └── pdf/
│       └── renderer_test.go
├── test/
│   ├── e2e/
│   │   └── cli_test.go
│   ├── integration/
│   │   └── pipeline_test.go
│   ├── golden/
│   │   └── [golden test files]
│   ├── fixtures/
│   │   └── [test input files]
│   └── benchmark/
│       └── performance_test.go
```

11.2 Test Execution Commands

```
# Run all unit tests
go test ./...

# Run with coverage
go test -cover ./...

# Run specific package tests
go test ./internal/pkg/mdflux/converter/...

# Run benchmarks
go test -bench=. -benchmem ./test/benchmark/

# Run with race detector
go test -race ./...

# Run integration tests (requires Chrome)
go test -tags=integration ./test/integration/...

# Run E2E tests (requires Chrome + built binary)
go build -o mdflux ./cmd/mdflux && go test -tags=e2e ./test/e2e/

# Run fuzz tests
go test -fuzz=Fuzz -fuzztime=60s ./...

# Generate coverage report
go test -coverprofile=coverage.out -covermode=atomic ./...
```

```
go tool cover -html=coverage.out -o coverage.html

# Security scan
gosec ./...
govulncheck ./...
```

11.3 Test Data Management

- Keep test fixtures small and focused
- Use `//go:embed` for test data files
- Clean up temporary files in test teardown
- Use `t.TempDir()` for temporary directories
- Normalize dynamic content in golden file comparisons

---

12. Coverage Targets

Package	Line Coverage Target
`config`	90%
`converter`	85%
`mermaid`	80%
`pdf`	75%
Overall	80%

---

13. Test Prioritization

High Priority (Must Have)

- UC-CNV-* (Core conversion)
- UC-E2E-001 through UC-E2E-005 (Basic CLI)
- UC-INT-* (Integration tests)
- UC-SEC-* (Security tests)

Medium Priority (Should Have)

- UC-EXT-* (Extension tests)
- UC-EDGE-001 through UC-EDGE-010 (Common edge cases)

- UC-PERF-001 through UC-PERF-005 (Performance benchmarks)

Low Priority (Nice to Have)

- UC-EDGE-020+ (Rare edge cases)
- UC-PERF-010+ (Concurrency/stress tests)
- UC-REG-* (Regression tests - build over time)

---

14. Maintenance

- Review and update this strategy quarterly
- Add regression tests for every bug fix
- Update golden files when intentional output changes occur
- Monitor test execution time and optimize slow tests
- Archive deprecated test cases rather than deleting